a7bb13ce-b997-401b-a39c-8205f45b8baf

<prev_user>

<request>

You are working on a task in the 'melanchall/drywetmidi' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/drywetmidi/drywetmidi (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-05-04T06:37:21.028+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>
You can see in macOS implementation that I use PROPERTY_VALUE_BUFFER_SIZE constant. Can I don't rely on fixed size and know from CFStringRef what buffer size I need? Because PROPERTY_VALUE_BUFFER_SIZE can be too small. I want to create buffer of size which will be enough for property value.

Also looks like those buffers are leaks, because I never free them. I need these strings from C# side. Can I just call Marshal.FreeHGlobal on buffer IntPtr after I've converted pointer to string? Or it's wrong to free memory in C# in case of memory was allocated from C++ (native code)?
</problem_statement>

First, determine whether the <problem_statement> requires code changes, an explanation, a plan, or analysis. Then follow the appropriate path:

If the task requires code changes:

**If the task is a question or only requires creating a plan or

analysis:** respond directly without making code changes.

<prev_assistant>

I now have everything I need. This is a pure analysis/question — no code changes required. Here are the answers:

Question 1: How to get the correct buffer size from `CFStringRef` instead of using `PROPERTY_VALUE_BUFFER_SIZE`

Yes, you can determine the exact required size before allocating. The standard Apple API for this is `CFStringGetMaximumSizeForEncoding`:

```cpp
CFIndex length =
CFStringGetLength(stringRef);                                    // UTF-16
code units
CFIndex maxSize = CFStringGetMaximumSizeForEncoding(length,
kCFStringEncodingUTF8) + 1; // +1 for null terminator
char* bC#):** Cache the string in the native struct (which the current code already does in `parentDeviceName`). Since `DeviceInfoBase` owns the buffer and the struct is deleted when the device is released, no explicit free is needed from C# — the C++ destructor handles it. This is already what happens in the current implementation for the `parentDeviceName` field stored in `DeviceInfoBase`.

**Option B (if you want C# to own the buffer):** Allocate with `CoTaskMemAlloc` in C++:
```cpp
// In C++:
size_t len = strlen(buffer) + 1;
char* coTaskBuffer = (char*)CoTaskMemAlloc(len);
memcpy(coTaskBuffer, buffer, len);
delete[] buffer;
*value = coTaskBuffer;
```
Then from C# you can safely call `Marshal.FreeCoTaskMem(pointer)` after reading the string. This crosses the allocator boundary safely because `CoTaskMemAlloc`/`CoTaskMemFree` and `Marshal.FreeCoTaskMem` all go through the same COM task allocator.

Recommendation for this codebase: Keep Option A. The native `DeviceInfoBase` struct already caches the result in `parentDeviceName`,
```

and `DeleteInputDeviceInfo`/`DeleteOutputDeviceInfo` call `delete info` which frees the struct and its members. No changes needed on the C# side — the C++ code owns and frees these strings through its own lifetime management, and C# only reads them via `Marshal.PtrToStringAnsi` (which copies the data into a managed `string`, never owns the native pointer).

<current>

<request>

You are working on a task in the 'melanchall/drywetmidi' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/drywetmidi/drywetmidi (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-05-04T16:55:55.164+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>
Look into dev-parent branch again at this instruction in native Windows file:

```
        auto additionalInfo =
winrt::Windows::Devices::Enumeration::DeviceInformation::CreateF
romIdAsync(
            parentId,
            winrt::single_threaded_vector
```

</problem_statement>

First, determine whether the <problem_statement> requires code changes, an explanation, a plan, or analysis. Then follow the appropriate path:

If the task requires code changes:

If the task is a question or only requires creating a plan or analysis: respond directly without making code changes.

<prompt>

Look into dev-parent branch again at this instruction in native Windows file:

```
        auto additionalInfo =
winrt::Windows::Devices::Enumeration::DeviceInformation::CreateFromI
dAsync(
            parentId,
            winrt::single_threaded_vector<winrt::hstring>(
            {
                manufacturerPropertyName,
                modelPropertyName
            })).get();
```

I get SEHException on get. Stack trace:

```
        [System.Runtime.InteropServices.SEHException thrown]
        ntdll.dll!00007ffae6f81b84()    Unknown
        KernelBase.dll!00007ffae393b1bf()       Unknown
        System.Private.CoreLib.dll!00007ff9d0e9a545()   Unknown
        ntdll.dll!00007ffae6f862bf()    Unknown
        ntdll.dll!00007ffae6e2e603()    Unknown
        ntdll.dll!00007ffae6ef8cc6()    Unknown
        ntdll.dll!00007ffae6f8623f()    Unknown
        ntdll.dll!00007ffae6e32397()    Unknown
        ntdll.dll!00007ffae6f85b7e()    Unknown
        KernelBase.dll!00007ffae39c79da()       Unknown
        vcruntime140.dll!00007ffac1a15277()     Unknown
        Melanchall_DryWetMidi_Native.dll!winrt::throw_hresult(const
winrt::hresult result, const winrt::impl::slim_source_location &
sourceInformation) Line 5042    C++
        [Inline Frame] Melanchall_DryWetMidi_Native.dll!
winrt::check_hresult(const winrt::hresult) Line 5112    C++
        [Inline Frame] Melanchall_DryWetMidi_Native.dll!
winrt::impl::consume_Windows_Foundation_IAsyncOperation<winrt::Windo
ws::Foundation::IAsyncOperation<winrt::Windows::Devices::Enumeration
::DeviceInformation>,winrt::Windows::Devices::Enumeration::DeviceInf
ormation>::GetResults() Line 367        C++
        Melanchall_DryWetMidi_Native.dll!
winrt::impl::wait_get<winrt::Windows::Foundation::IAsyncOperation<wi
nrt::Windows::Devices::Enumeration::DeviceInformation>>(const
winrt::Windows::Foundation::IAsyncOperation<winrt::Windows::Devices:
:Enumeration::DeviceInformation> & async) Line 5140     C++
        [Inline Frame] Melanchall_DryWetMidi_Native.dll!
winrt::impl::consume_Windows_Foundation_IAsyncOperation<winrt::Windo
ws::Foundation::IAsyncOperation<winrt::Windows::Devices::Enumeration
```

```
::DeviceInformation>,winrt::Windows::Devices::Enumeration::DeviceInf
ormation>::get() Line 5272      C++
>       Melanchall_DryWetMidi_Native.dll!
GetParentDeviceInfo_Win(DeviceInfoBase * deviceInfo, const wchar_t *
* id, const wchar_t * * name, const wchar_t * * manufacturer, const
wchar_t * * model, int * errorCode) Line 702    C++
```